

Progetto 1: Implementazione di algoritmi paralleli per il simulare la crescita cristallina

Diffusion-limited aggregation (DLA) è un processo di formazione di cristalli nel quale le particelle si muovono in uno spazio 2D con moto browniano (cioè in modo casuale) e si combinano tra loro quando si toccano. DLA può essere simulato utilizzando una griglia 2D in cui ogni cella può essere occupata da uno o più particelle in movimento. Una particella diventa parte di un cristallo (e si ferma) quando si trova in prossimità di un cristallo già formato. I parametri di base della simulazione sono la dimensione della griglia 2D, il numero iniziale di particelle, il numero di iterazioni e il "seme" cristallino iniziale. Implementare l'algoritmo di DLA utilizzando 2 tra i seguenti approcci: MPI, PThread/OpenMP, CUDA.

Verificare la correttezza degli algoritmi implementati, confrontando i risultati con quelli ottenuti da una versione single-thread. Valutare le prestazioni degli algoritmi sviluppati in termini di speed-up ed efficienza al variare del numero di processi/thread e delle dimensioni del problema (numero di particelle, numero di iterazioni e dimensioni della griglia).

Il progetto è dimensionato per essere realizzato da un gruppo composto da due studenti.

La consegna del progetto (almeno una settimana prima dell'orale) consiste in:

(a) tutti i sorgenti (opportunamente commentati) necessari per il funzionamento;

(b) Una relazione contenente:

- La descrizione dettagliata dell'architettura dell'applicazione e delle scelte progettuali effettuate, opportunamente motivate.
- La descrizione delle eventuali limitazioni riscontrate;
- I risultati in termini di prestazioni opportunamente commentati;

Il giorno dell'orale è necessario preparare una presentazione powerpoint ed una demo del progetto.

Progetto 2: Implementazione di algoritmi paralleli per il calcolo della similarità tra documenti

Dato un insieme di documenti **D**, l'algoritmo MinHash consente di controllare la similarità dei documenti appartenenti a **D**. Sviluppare l'algoritmo utilizzando 2 tra i seguenti approcci: MPI, PThread/OpenMP, CUDA.

Oltre a verificare la correttezza degli algoritmi implementati (ad esempio confrontando i risultati con quelli ottenuti da una versione single-thread), valutare le prestazioni degli algoritmi sviluppati in termini di speed-up ed efficienza al variare del numero di processi/thread e delle dimensioni del problema (numero di documenti **D**).

Il progetto è dimensionato per essere realizzato da un gruppo composto da due studenti.

La consegna del progetto (almeno una settimana prima dell'orale) consiste in:

(a) tutti i sorgenti (opportunamente commentati) necessari per il funzionamento;

(b) Una relazione contenente:

- La descrizione dettagliata dell'architettura dell'applicazione e delle scelte progettuali effettuate, opportunamente motivate.
- La descrizione delle eventuali limitazioni riscontrate;
- I risultati in termini di prestazioni opportunamente commentati;

Il giorno dell'orale è necessario preparare una presentazione powerpoint ed una demo del progetto.

Riferimenti bibliografici:

[1] <https://en.wikipedia.org/wiki/MinHash>

[2] A. Broder, "Identifying and Filtering Near-Duplicate Documents",

<http://cs.brown.edu/courses/cs253/papers/nearduplicate.pdf>

[3] <http://matthewcasperson.blogspot.com/2013/11/minhash-for-dummies.html>

[4] Minhash <http://infolab.stanford.edu/~ullman/mmds/ch3.pdf>

Progetto 3: Implementazione dell'algoritmo di clustering K-means in parallelo

Dato un insieme O di N oggetti, ognuno con i attributi, implementare l'algoritmo di clustering di k-means [1] utilizzando 2 tra i seguenti approcci: MPI, PThread/OpenMP, CUDA.

Oltre a verificare la correttezza degli algoritmi implementati (ad esempio confrontando i risultati con quelli ottenuti da una versione single-thread), valutare le prestazioni degli algoritmi sviluppati in termini di speed-up ed efficienza al variare del numero di processi/thread, del numero di oggetti N e del numero di attributi i .

Il progetto è dimensionato per essere realizzato da un gruppo composto da due studenti.

La consegna del progetto (almeno una settimana prima dell'orale) consiste in:

(a) tutti i sorgenti (opportunamente commentati) necessari per il funzionamento;

(b) Una relazione contenente:

- La descrizione dettagliata dell'architettura dell'applicazione e delle scelte progettuali effettuate, opportunamente motivate.

- La descrizione delle eventuali limitazioni riscontrate;

- I risultati in termini di prestazioni opportunamente commentati;

Il giorno dell'orale è necessario preparare una presentazione powerpoint ed una demo del progetto.

Riferimenti bibliografici:

[1] <https://it.wikipedia.org/wiki/K-means>

Progetto 4: Implementazione di un algoritmo per la soluzione del problema degli n-corpi

Dato un insieme $\mathbf{O}$ di N oggetti, di cui siano noti massa, posizione e velocità iniziale, implementare un algoritmo che simula l'evoluzione temporale del sistema soggetto alla forza di gravitazione universale (problema degli n-corpi [1]). Utilizzare 2 tra i seguenti approcci: MPI, PThread/OpenMP, CUDA. Oltre al metodo esaustivo [2, ch.6], implementare il metodo di Barnes-Hut [3]. Verificare la correttezza degli algoritmi implementati (ad esempio confrontando i risultati con quelli ottenuti da una versione single-thread), valutare le prestazioni degli algoritmi sviluppati in termini di speed-up ed efficienza al variare del numero di processi/thread e del numero di oggetti N .

Il progetto è dimensionato per essere realizzato da un gruppo composto da due/tre studenti.

La consegna del progetto (almeno una settimana prima dell'orale) consiste in:

(a) tutti i sorgenti (opportunamente commentati) necessari per il funzionamento;

(b) Una relazione contenente:

- La descrizione dettagliata dell'architettura dell'applicazione e delle scelte progettuali effettuate, opportunamente motivate.
- La descrizione delle eventuali limitazioni riscontrate;
- I risultati in termini di prestazioni opportunamente commentati;

Il giorno dell'orale è necessario preparare una presentazione powerpoint ed una demo del progetto.

Riferimenti bibliografici:

[1] https://it.wikipedia.org/wiki/Problema_degli_n-corpi

[2] Pacheco, Peter. An introduction to parallel programming. Elsevier, 2011.

[3] https://it.wikipedia.org/wiki/Algoritmo_di_Barnes-Hut

Progetto 5: Implementazione di collettive MPI su dati quantizzati

Le operazioni collettive (ad esempio, allreduce e alltoall) rappresentano una larga parte del tempo di esecuzione durante l'addestramento di modelli di machine learning [1]. Per ridurre l'impatto di queste comunicazioni, diverse tecniche di quantizzazione possono essere utilizzate per comprimere i dati trasmessi. Quando si usa la quantizzazione, ciascun elemento viene rappresentato su un numero minore di bit, prima che venga trasmesso all'interno di un'operazione collettiva. Gli approcci di quantizzazione devono essere progettati con cautela. Infatti, da un lato l'errore introdotto dalla quantizzazione deve essere tenuto sotto controllo, mentre dall'altro lato il processo di quantizzazione stesso non deve essere troppo costoso da un punto di vista computazionale (lavorare con elementi più piccoli di un byte potrebbe essere inefficiente).

Il progetto può essere scalato in complessità in base al numero di studenti. Di base prevede:

- L'implementazione di collettive (principalmente allreduce) su dati quantizzati, utilizzando algoritmi di quantizzazione esistenti (da decidere all'inizio del progetto), e potenzialmente sviluppandone di nuovi. La allreduce andrà implementata ad anello sia con struttura ad albero (recursive doubling/halving)
- La quantizzazione/dequantizzazione andrà effettuata in modo efficiente, potenzialmente usando OpenMP
- L'algoritmo di quantizzazione e il numero di bit da utilizzare per ogni elemento devono poter essere specificati tramite un parametro e/o variabile d'ambiente
- Il codice deve essere utilizzabile in modo trasparente in applicazioni MPI esistenti (vedere libreria PMPI (<https://www.open-mpi.org/faq/?category=perftools>), che intercetta le chiamate ad MPI in modo da introdurre codice aggiuntivo)
- L'analisi delle prestazioni e dell'errore introdotto dalla quantizzazione

La consegna del progetto (almeno una settimana prima dell'orale) consiste in:

- Tutti i sorgenti (opportunosamente commentati) necessari per il funzionamento;
- Una relazione contenente:
 - La descrizione dettagliata dell'architettura dell'applicazione e delle scelte progettuali effettuate, opportunosamente motivate.
 - La descrizione delle eventuali limitazioni riscontrate;
 - I risultati in termini di prestazioni opportunosamente commentati;

Il giorno dell'orale è necessario preparare una presentazione powerpoint ed una demo del progetto.

[1] Jie Yang et al., "Training Deep Learning Recommendation Model with Quantized Collective Communications"

Progetto 6: Implementazione di collettive MPI su dati sparsi

Le operazioni collettive (ad esempio, allreduce e alltoall) rappresentano una larga parte del tempo di esecuzione durante l'addestramento di modelli di machine learning. Per ridurre l'impatto di queste comunicazioni, diverse tecniche di sparsificazione possono essere utilizzate per comprimere i dati trasmessi. Quando si usa la sparsificazione, alcuni elementi vengono scartati (verranno considerati uguali a 0), prima di venir trasmessi all'interno di un'operazione collettiva. Poichè molti elementi saranno uguali a 0, l'operazione collettiva può quindi trasmettere soltanto i valori diversi da 0, riducendo il volume dei dati comunicati. I dati andranno quindi rappresentati come dei vettori sparsi, utilizzando vari formati esistenti. Nel caso della allreduce, questi vettori sparsi andranno aggregati (ad esempio sommati), e questo aggiunge complessità in quanto ad ogni step i dati diventeranno sempre meno densi, arrivando quindi ad un punto in cui diventa più conveniente rappresentarli come dati densi anzichè sparsi [1].

Il progetto può essere scalato in complessità in base al numero di studenti. Di base prevede:

- L'implementazione di collettive (principalmente allreduce) su dati sparsi, utilizzando algoritmi di sparsificazione esistenti (da decidere all'inizio del progetto), e potenzialmente sviluppandone di nuovi. La allreduce andrà implementata ad anello sia con struttura ad albero (recursive doubling/halving)
- Bisogna considerare la possibilità che ad ogni step della allreduce i dati cambiano la loro densità, e che quindi potrebbe essere opportuno passare a formati diversi [2, Fig. 5]
- L'algoritmo di sparsificazione e la percentuale di elementi da settare a zero devono poter essere specificati tramite un parametro e/o variabile d'ambiente
- Il codice deve essere utilizzabile in modo trasparente in applicazioni MPI esistenti (vedere libreria PMPI (<https://www.open-mpi.org/faq/?category=perftools>), che intercetta le chiamate ad MPI in modo da introdurre codice aggiuntivo)
- L'analisi delle prestazioni e dell'errore introdotto dalla sparsificazione

La consegna del progetto (almeno una settimana prima dell'orale) consiste in:

- Tutti i sorgenti (opportunamente commentati) necessari per il funzionamento;
- Una relazione contenente:
 - La descrizione dettagliata dell'architettura dell'applicazione e delle scelte progettuali effettuate, opportunamente motivate.
 - La descrizione delle eventuali limitazioni riscontrate;
 - I risultati in termini di prestazioni opportunamente commentati;

Il giorno dell'orale è necessario preparare una presentazione powerpoint ed una demo del progetto.

[1] Renggli et al., "SparCML: High-Performance Sparse Communication for Machine Learning"

[2] Hoefler et al., "Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks"